Winston Shih (WXS190012)
Satyam Garg (SXG210250)
CS 4337.004

CS 4337 HW 3

1.

WXS190012-SXG210250-HW3-Code-Q1

```
%Winston Shih (WXS190012)
%Satyam Garg (SXG210250)
%CS 4337.004 HW 3 Question 1
% Checks if a number is prime by seeing if  it does not have a divisor starting with 2.
is_prime(N) :-
    N > 1,
    \+ has_divisor(N, 2).
% Check if N has any divisor from D up to sqrt(N).
has_divisor(N, D) :-
    D * D =< N,
    N mod D =:= 0.
% Searches for a divisor of N up to square root of N.
has_divisor(N, D) :-
    D * D =< N, D1 is D + 1, has_divisor(N, D1).
% Check if a number is both prime and greater than 5
is_prime_gt5(N) :-
    N > 5, is_prime(N).
% Sum all numbers in a list that are prime and greater than 5
sum_prime_greater_than_5([], 0).
% Checks to see if H is a prime greater than 5.
sum_prime_greater_than_5([H|T], Sum) :-
    is_prime_gt5(H),
    sum_prime_greater_than_5(T, RestSum),
    Sum is H + RestSum.
%Checks to see if H is not a prime greater than 5.
sum_prime_greater_than_5([H|T], Sum) :-
    \+ is_prime_gt5(H),
    sum_prime_greater_than_5(T, Sum).
```

sum_prime_greater_than_5([7,8,11,13],X).

X = 31

Next | 10 | 100 | 1,000 | Stop

```
?- sum_prime_greater_than_5( [7,8,11,13],X).
```

2.

SWISH File Edit Examples Help 189 users online Search

WXS190012-SXG210250-HW3-Code-Q2a

```
%Winston Shih (WXS190012)
%Satyam Garg (SXG210250)
%CS 4337.004 HW 3 Question 2 Bubble Sort
bubble_sort(List, SortedList) :-                % Main bubble sort method.
    is_sorted(List),                            % Checks if List is already sorted.
    SortedList = List.                          % If List is sorted, then SortedList is set to List.
bubble_sort(List, SortedList) :-                % Alternative bubble sort method (if List is not sorted).
    \+ is_sorted(List),                         % Verifies if List is not sorted.
    bubble_pass(List, TempList),                % Performs a bubble sort through List with a TempList.
    bubble_sort(TempList, SortedList).          % Uses recursion calls to bubble sort List until fully sorted.
bubble_pass([], []).                            % Base case: The list is empty.
bubble_pass([X], [X]).                          % Base case: The list has one element
bubble_pass([X, Y | Rest], [Y, X | TempRest]) :- % Case: List where first element is greater than second.
    X > Y,                                      % Checks if first element is greater than second.
    bubble_pass([Y | Rest], TempRest).          % Recursively bubble pass X with rest of list for comparison.
bubble_pass([X, Y | Rest], [X | TempRest]) :-   % Case: List where first element is smaller than second.
    X =< Y,                                     % Checks if first element is smaller than second.
    bubble_pass([Y | Rest], TempRest).          % Recursively bubble pass Y with rest of list for comparison.
is_sorted([]).                                  % Base case: Checks to see if empty list is sorted.
is_sorted([_]).                                 % Base case: Checks to see if one element list is sorted.
is_sorted([X, Y | Rest]) :-                     % Case: Check if 2 or more element list is sorted.
    X =< Y,                                     % Verify first element is less than or equal to second.
    is_sorted([Y | Rest]).                      % Recursively check if rest of 2 or more element list is sorted.
```

bubble_sort([3,4,5], X).

X = [3, 4, 5]

false

```
?- bubble_sort([3,4,5], X).
```

SWISH File Edit Examples Help 193 users online Search

WXS190012-SXG210250-HW3-Code-Q2b

```
%Winston Shih (WXS190012)
%Satyam Garg (SXG210250)
%CS 4337.004 HW 3 Question 2 Quick Sort
quicksort([], []).                    % Base case: Empty list is already sorted.
quicksort([X|Xs], Ys) :-              % Sort a list with X (first element) and Xs (remaining elements) into Ys.
    partition(Xs, X, Left, Right),    % Partition Xs into Left and Right subarrays with X as a pivot.
    quicksort(Left, Ls),              % Recursively sort the Left partition of Xs and store in Ls.
    quicksort(Right, Rs),             % Recursively sort the Right partition of Xs and store in Rs.
    append(Ls, [X|Rs], Ys).           % Combine: Combine Ls (left partition) and [X|Rs] (pivot and right partition).
                                      % Store combination into Ys.
partition([], _, [], []).             % Base case: Partition an empty list into two empty lists.
partition([X|Xs], Y, [X|Ls], Rs) :-   % Case: Partition a list when element X is less than pivot Y.
    X =< Y,                           % Check if X is less than or equal to pivot Y
    partition(Xs, Y, Ls, Rs).         % If true, then recursively partition Xs and X goes to left partition
partition([X|Xs], Y, Ls, [X|Rs]) :-   % Case: Partition a list when element X is greater than pivot Y.
    X > Y,                            % Check if element X is greater than pivot Y.
    partition(Xs, Y, Ls, Rs).         % If true, then recursively Xs and X goes to right partition
append([], Ys, Ys).                   % Base case: Appending empty list to Ys results in list of Ys.
append([X|Xs], Ys, [X|Zs]) :-         % Recursive case: Appending [X|Xs] to Ys results in list [X|Zs].
    append(Xs, Ys, Zs).               % Recursively append Xs and Ys to get Zs
```

quicksort([2,1,4], X).

X = [1, 2, 4]

false

?- quicksort([2,1,4], X).

3.

WXS190012-SXG210250-HW3-Code-Q3

```
%Winston Shih (WXS190012)
%Satyam Garg (SXG210250)
%CS 4337.004 HW 3 Question 3
% Base case: empty list has 0 elements
elements([], 0).
% Case 1: Head is a list - recursively count its elements and add to tail count
elements([H|T], Count) :-
    is_list(H),
    elements(H, HCount),
    elements(T, TCount),
    Count is HCount + TCount.
% Case 2: Head is an atom/number - count it as 1 and add to tail count
elements([H|T], Count) :-
    \+ is_list(H),
    elements(T, TCount),
    Count is 1 + TCount.
```

elements([b, [a, [d, c], e]], X).

X = 5

Next 10 100 1,000 Stop

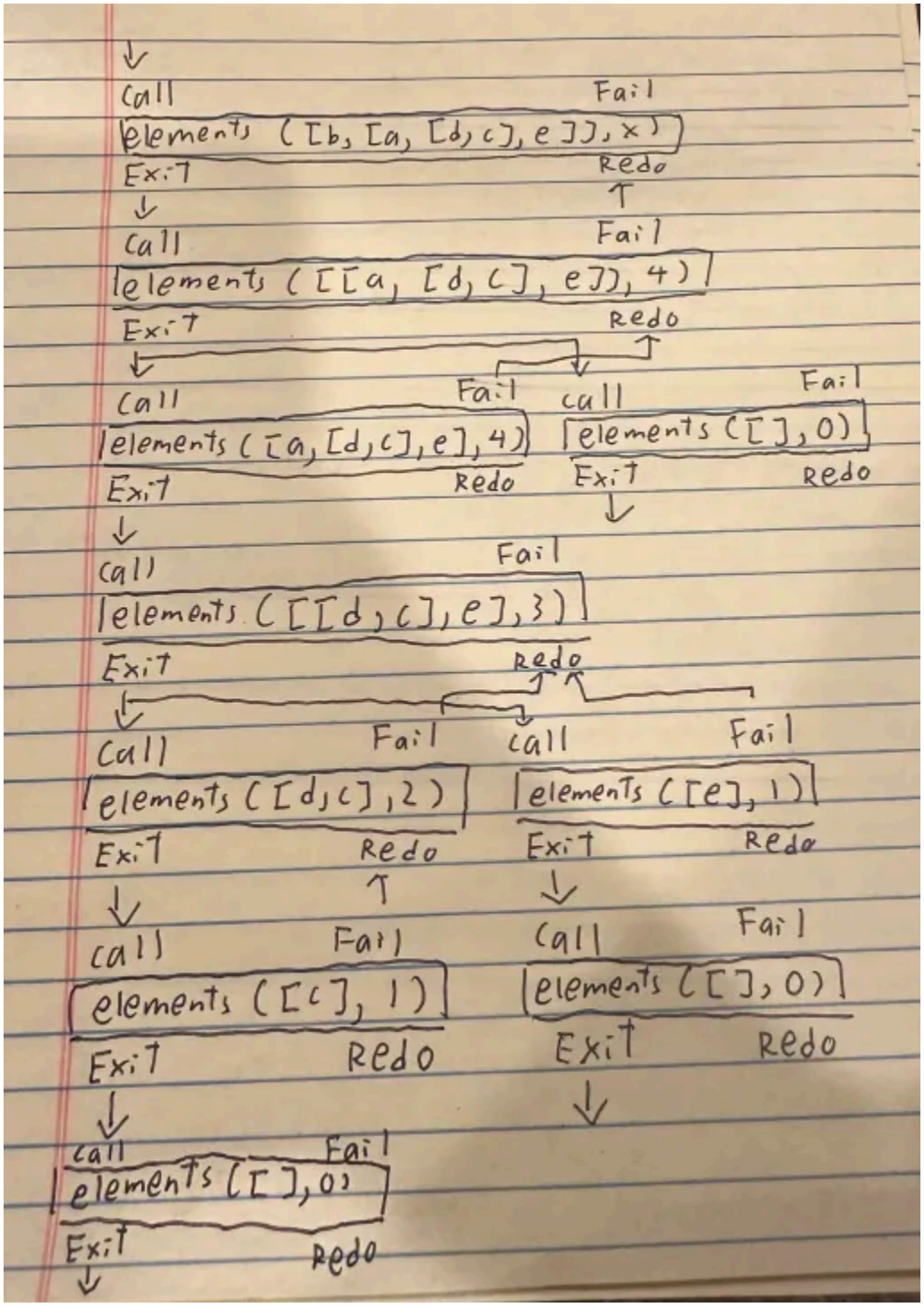
?- elements([b, [a, [d, c], e]], X).

Subgoal 1	Subgoal 2	Action	Result
(1)	2	Call	elements([b, [a, [d, c], e]], X) H=b, T=[[a, [d, c], e]] is_list(b)? NO → Fail clause 2
(1)	3	Redo	elements([b, [a, [d, c], e]], X) H=b, T=[[a,

			[d, c], e]] + is_list(b)? YES
(2)	2	Call	elements([[a, [d, c], e]], CountT) H=[a, [d, c], e], T=[] is_list([a, [d, c], e])? YES
(3)	2	Call	elements([a, [d, c], e], CountH) H=a, T=[[d, c], e] is_list(a)? NO → Fail clause 2
(3)	3	Redo	elements([a, [d, c], e], CountH) H=a, T=[[d, c], e] + is_list(a)? YES
(4)	2	Call	elements([[d, c], e], CountT) H=[d, c], T=[e] is_list([d, c])? YES
(5)	2	Call	elements([d, c], CountH) H=d, T=[c] is_list(d)? NO → Fail clause 2
(5)	3	Redo	elements([d, c], CountH) H=d, T=[c] + is_list(d)? YES
(6)	2	Call	elements([c], CountT) H=c, T=[] is_list(c)? NO → Fail clause 2
(6)	3	Redo	elements([c], CountT) H=c, T=[] + is_list(c)? YES
(7)	1	Call	elements([], CountT)

(7)	1	Exit	elements([], 0) ← CountT=0
(6)	3	Exit	elements([c], 1) ← CountT=1 Count is 0 + 1 = 1
(5)	3	Exit	elements([d, c], 2) ← CountH=2 Count is 1 + 1 = 2
(8)	2	Call	elements([e], CountT) H=e, T=[] is_list(e)? NO → Fail clause 2
(8)	3	Redo	elements([e], CountT) H=e, T=[] + is_list(e)? YES
(9)	1	Call	elements([], CountT)
(9)	1	Exit	elements([], 0) ← CountT=0
(8)	3	Exit	elements([e], 1) ← CountT=1 Count is 0 + 1 = 1
(4)	2	Exit	elements([[d, c], e], 3) ← CountT=3 Count is 2 + 1 = 3
(3)	3	Exit	elements([a, [d, c], e], 4) ← CountH=4 Count is 3 + 1 = 4
(10)	1	Call	elements([], CountT)
(10)	1	Exit	elements([], 0) ← CountT=0
(2)	2	Exit	elements([[a, [d, c], e]], 4) ← CountT=4 Count is 4 + 0 = 4

(1)	3	Exit	elements([b, [a, [d, c], e]], 5) ← X=5 Count is 4 + 1 = 5



4.

SWISH File Edit Examples Help

180 users online

WXS190012-SXG210250-HW3-Code-Q4

```
%Winston Shih (WXS190012)
%Satyam Garg (SXG210250)
%CS 4337.004 HW 3 Question 4
% Genders of family members
male(winston).
male(billy).
male(jack).
male(tom).
male(leo).
male(alex).
male(mike).
male(george).
male(harry).
male(peter).
male(connor).
female(fiona).
female(audrey).
female(ella).
female(nancy).
female(lisa).
female(sarah).
female(mary).
female(julia).
female(susan).
% Parental relationships.
parent(billy, winston).
parent(fiona, winston).
parent(billy, audrey).
parent(fiona, audrey).
parent(jack, fiona).
```

mother(X,winston).

X = fiona

father(X,winston).

X = billy

Next 10 100 1,000 Stop

count_brothers(winston, X).

X = 1

sister(X,winston).

X = audrey

Next 10 100 1,000 Stop

brother(X,winston).

X = mike

Next 10 100 1,000 Stop

?- grandfather(X,winston).

Examples History Solutions

WXS190012-SXG210250-HW3-Code-Q4

```
parent(ella, fiona).
parent(tom, billy).
parent(lisa, billy).
parent(jack, leo).
parent(ella, leo).
parent(alex, nancy).
parent(audrey, nancy).
parent(billy, sarah).
parent(fiona, sarah).
parent(billy, mike).
parent(fiona, mike).
parent(george, jack).
parent(harry, george).
parent(tom, mary).
parent(audrey, julia).
parent(alex, julia).
parent(audrey, peter).
parent(jack, susan).
parent(ella, susan).
parent(susan, connor).
% Parent rules
mother(X, Y) :- parent(X, Y), female(X).
father(X, Y) :- parent(X, Y), male(X).
% Sibling rules.
sibling(X, Y) :-
    father(F, X), father(F, Y),
    mother(M, X), mother(M, Y),
    X \== Y.
%Sister and brother rules.
sister(X, Y) :- sibling(X, Y), female(X).
```

grandfather(X,winston).

X = jack

X = tom

false

grandmother(X,winston).

X = ella

X = lisa

false

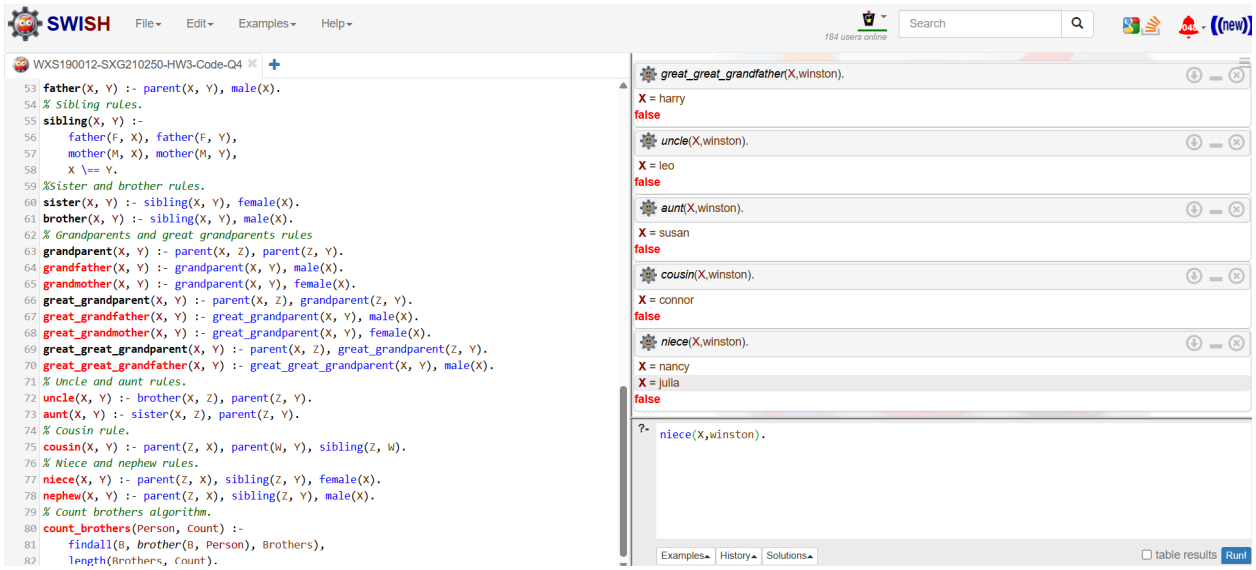
great_grandfather(X,winston).

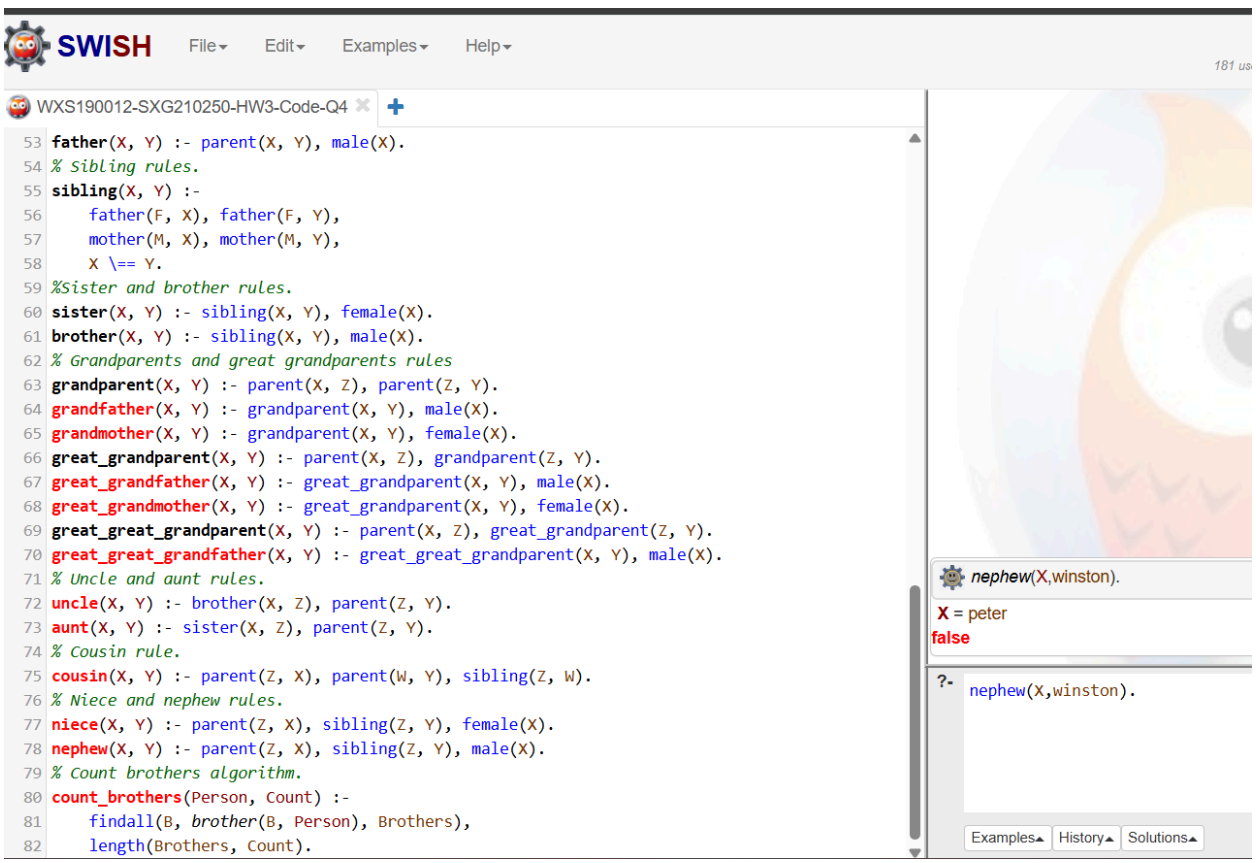
X = george

false

?- great_great_grandfather(X,winston).

Examples History Solutions





Queries:

mother(X,winston).

father(X,winston).

count_brothers(winston,X).

sister(X,winston).

brother(X,winston).

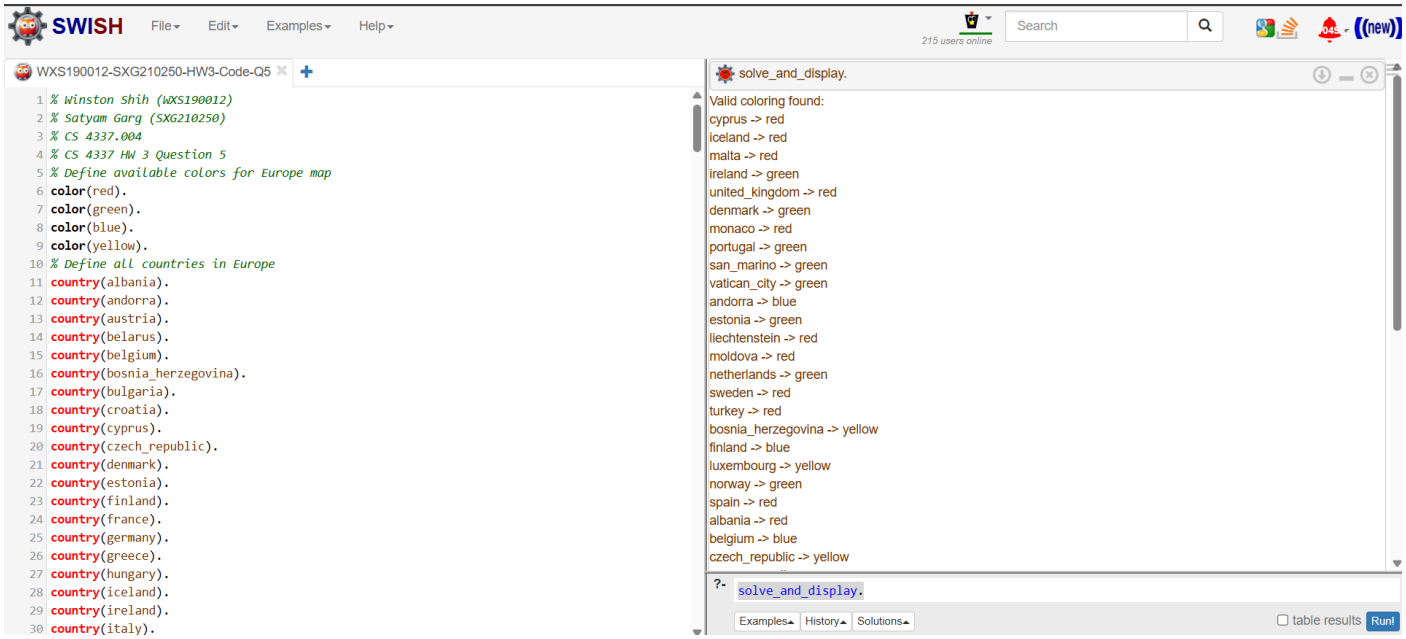
```
grandfather(X,winston).
grandmother(X,winston).
great_grandfather(X,winston).
great_great_grandfather(X,winston).
uncle(X,winston).
aunt(X,winston).
cousin(X,winston).
niece(X,winston).
nephew(X,winston).
```

Who is your mother?
Fiona
Who is your father?
Billy
How many brothers?
1
Who is/are your sisters?
Audrey.
Who is/are your brothers?
Mike.
Who are your grandfathers?
Jack and Tom.
Who are your grandmothers?
Ella and Lisa.
Who is your great grand father?
George.
Who is your great great grand father?
Harry
Who is/are your uncle/s?
Leo.
Who is/are your aunt/s?
Susan.
Who is/are your cousin/s?
Connor.
Who is/are your niece/s?
Nancy and Julia.
Who is/are your nephew/s?
Peter.

5.

Output of program:
cyprus -> red
iceland -> red
malta -> red
ireland -> green
united_kingdom -> red
denmark -> green
monaco -> red
portugal -> green
san_marino -> green
vatican_city -> green
andorra -> blue
estonia -> green
liechtenstein -> red
moldova -> red
netherlands -> green
sweden -> red
turkey -> red
bosnia_herzegovina -> yellow
finland -> blue
luxembourg -> yellow

norway -> green
spain -> red
albania -> red
belgium -> blue
czech_republic -> yellow
greece -> yellow
kosovo -> yellow
latvia -> blue
lithuania -> green
slovenia -> yellow
belarus -> yellow
bulgaria -> blue
croatia -> green
montenegro -> blue
north_macedonia -> green
romania -> yellow
slovakia -> red
switzerland -> blue
italy -> red
hungary -> blue
poland -> blue
ukraine -> green
austria -> green
france -> green
russia -> red
serbia -> red
germany -> red



SWISH File Edit Examples Help

217 users online

Search

WXS190012-SXG210250-HW3-Code-Q5

```
country(kosovo).
country(latvia).
country(liechtenstein).
country(lithuania).
country(luxembourg).
country(malta).
country(moldova).
country(monaco).
country(montenegro).
country(netherlands).
country(north_macedonia).
country(norway).
country(poland).
country(portugal).
country(romania).
country(russia).
country(san_marino).
country(serbia).
country(slovakia).
country(slovenia).
country(spain).
country(sweden).
country(switzerland).
country(turkey).
country(ukraine).
country(united_kingdom).
country(vatican_city).
% Countries adjacent to Albania.
adjacent(albania, montenegro).
adjacent(albania, kosovo).
```

```
albania -> red
belgium -> blue
czech_republic -> yellow
greece -> yellow
kosovo -> yellow
latvia -> blue
lithuania -> green
slovenia -> yellow
belarus -> yellow
bulgaria -> blue
croatia -> green
montenegro -> blue
north_macedonia -> green
romania -> yellow
slovakia -> red
switzerland -> blue
italy -> red
hungary -> blue
poland -> blue
ukraine -> green
austria -> green
france -> green
russia -> red
serbia -> red
germany -> red
true
```

?- solve_and_display.

Examples History Solutions table results Run!

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
adjacent(albania, north_macedonia).
adjacent(albania, greece).
% Countries adjacent to Andorra.
adjacent(andorra, france).
adjacent(andorra, spain).
% Countries adjacent to Austria.
adjacent(austria, germany).
adjacent(austria, czech_republic).
adjacent(austria, slovakia).
adjacent(austria, hungary).
adjacent(austria, slovenia).
adjacent(austria, italy).
adjacent(austria, switzerland).
adjacent(austria, liechtenstein).
% Countries adjacent to Belarus.
adjacent(belarus, latvia).
adjacent(belarus, lithuania).
adjacent(belarus, poland).
adjacent(belarus, ukraine).
adjacent(belarus, russia).
% Countries adjacent to Belgium.
adjacent(belgium, france).
adjacent(belgium, luxembourg).
adjacent(belgium, germany).
adjacent(belgium, netherlands).
% Countries to Bosnia and Herzegovina
adjacent(bosnia_herzegovina, croatia).
adjacent(bosnia_herzegovina, serbia).
adjacent(bosnia_herzegovina, montenegro).
% Countries to Bulgaria.
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
adjacent(bulgaria, romania).
adjacent(bulgaria, serbia).
adjacent(bulgaria, north_macedonia).
adjacent(bulgaria, greece).
adjacent(bulgaria, turkey).
% Countries to Croatia.
adjacent(croatia, slovenia).
adjacent(croatia, hungary).
adjacent(croatia, serbia).
adjacent(croatia, bosnia_herzegovina).
adjacent(croatia, montenegro).
% Countries to Czech Republic.
adjacent(czech_republic, germany).
adjacent(czech_republic, poland).
adjacent(czech_republic, slovakia).
adjacent(czech_republic, austria).
% Countries adjacent to Denmark.
adjacent(denmark, germany).
% Countries adjacent to Estonia.
adjacent(estonia, russia).
adjacent(estonia, latvia).
% Countries adjacent to Finland.
adjacent(finland, sweden).
adjacent(finland, norway).
adjacent(finland, russia).
% Countries adjacent to France.
adjacent(france, spain).
adjacent(france, andorra).
adjacent(france, monaco).
adjacent(france, italy).
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
adjacent(france, switzerland).
adjacent(france, germany).
adjacent(france, luxembourg).
adjacent(france, belgium).
% Countries adjacent to Germany
adjacent(germany, denmark).
adjacent(germany, netherlands).
adjacent(germany, belgium).
adjacent(germany, luxembourg).
adjacent(germany, france).
adjacent(germany, switzerland).
adjacent(germany, austria).
adjacent(germany, czech_republic).
adjacent(germany, poland).
% Countries adjacent to Greece.
adjacent(greece, albania).
adjacent(greece, north_macedonia).
adjacent(greece, bulgaria).
adjacent(greece, turkey).
% Countries adjacent to Hungary.
adjacent(hungary, austria).
adjacent(hungary, slovakia).
adjacent(hungary, ukraine).
adjacent(hungary, romania).
adjacent(hungary, serbia).
adjacent(hungary, croatia).
adjacent(hungary, slovenia).
% Countries adjacent to Italy.
adjacent(italy, france).
adjacent(italy, switzerland).
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
adjacent(italy, austria).
adjacent(italy, slovenia).
adjacent(italy, san_marino).
adjacent(italy, vatican_city).
% Countries adjacent to Kosovo.
adjacent(kosovo, serbia).
adjacent(kosovo, north_macedonia).
adjacent(kosovo, albania).
adjacent(kosovo, montenegro).
% Countries adjacent to Latvia.
adjacent(latvia, estonia).
adjacent(latvia, russia).
adjacent(latvia, belarus).
adjacent(latvia, lithuania).
% Countries adjacent to Liechtenstein.
adjacent(liechtenstein, switzerland).
adjacent(liechtenstein, austria).
% Countries adjacent to Lithuania.
adjacent(lithuania, latvia).
adjacent(lithuania, belarus).
adjacent(lithuania, poland).
adjacent(lithuania, russia).
% Countries adjacent to Luxembourg.
adjacent(luxembourg, belgium).
adjacent(luxembourg, germany).
adjacent(luxembourg, france).
% Countries adjacent to Moldova.
adjacent(moldova, romania).
adjacent(moldova, ukraine).
% Countries adjacent to Monaco.
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
adjacent(monaco, france).
% Countries adjacent to Montenegro.
adjacent(montenegro, croatia).
adjacent(montenegro, bosnia_herzegovina).
adjacent(montenegro, serbia).
adjacent(montenegro, kosovo).
adjacent(montenegro, albania).
% Countries adjacent to Netherlands.
adjacent(netherlands, germany).
adjacent(netherlands, belgium).
% Countries adjacent to North Macedonia.
adjacent(north_macedonia, kosovo).
adjacent(north_macedonia, serbia).
adjacent(north_macedonia, bulgaria).
adjacent(north_macedonia, greece).
adjacent(north_macedonia, albania).
% Countries adjacent to Norway.
adjacent(norway, sweden).
adjacent(norway, finland).
adjacent(norway, russia).
% Countries adjacent to Poland.
adjacent(poland, germany).
adjacent(poland, czech_republic).
adjacent(poland, slovakia).
adjacent(poland, ukraine).
adjacent(poland, belarus).
adjacent(poland, lithuania).
adjacent(poland, russia).
% Countries adjacent to Portugal.
adjacent(portugal, spain).
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
% Romania
adjacent(romania, ukraine).
adjacent(romania, moldova).
adjacent(romania, bulgaria).
adjacent(romania, serbia).
adjacent(romania, hungary).
% European countries adjacent to Russia.
adjacent(russia, norway).
adjacent(russia, finland).
adjacent(russia, estonia).
adjacent(russia, latvia).
adjacent(russia, lithuania).
adjacent(russia, poland).
adjacent(russia, belarus).
adjacent(russia, ukraine).
% Countries adjacent to San Marino.
adjacent(san_marino, italy).
% Countries adjacent to Serbia.
adjacent(serbia, hungary).
adjacent(serbia, romania).
adjacent(serbia, bulgaria).
adjacent(serbia, north_macedonia).
adjacent(serbia, kosovo).
adjacent(serbia, montenegro).
adjacent(serbia, bosnia_herzegovina).
adjacent(serbia, croatia).
% Countries adjacent to Slovakia.
adjacent(slovakia, czech_republic).
adjacent(slovakia, poland).
adjacent(slovakia, ukraine).
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
adjacent(slovakia, hungary).
adjacent(slovakia, austria).
% Countries adjacent to Slovenia.
adjacent(slovenia, austria).
adjacent(slovenia, italy).
adjacent(slovenia, croatia).
adjacent(slovenia, hungary).
% Countries adjacent to Spain.
adjacent(spain, portugal).
adjacent(spain, france).
adjacent(spain, andorra).
% Countries adjacent to Sweden.
adjacent(sweden, norway).
adjacent(sweden, finland).
% Countries adjacent to Switzerland.
adjacent(switzerland, germany).
adjacent(switzerland, austria).
adjacent(switzerland, liechtenstein).
adjacent(switzerland, italy).
adjacent(switzerland, france).
% European countries adjacent to Turkey.
adjacent(turkey, greece).
adjacent(turkey, bulgaria).
% Countries adjacent to Ukraine.
adjacent(ukraine, poland).
adjacent(ukraine, slovakia).
adjacent(ukraine, hungary).
adjacent(ukraine, romania).
adjacent(ukraine, moldova).
adjacent(ukraine, russia).
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
adjacent(ukraine, belarus).
% Countries adjacent to United Kingdom.
adjacent(united_kingdom, ireland).
% Countries adjacent to Vatican City.
adjacent(vatican_city, italy).
% Optimized predicate to find a coloring using constraint checking during assignment
find_coloring(Colors) :-
    setof(Country, country(Country), Countries),
    sort_by_neighbors(Countries, SortedCountries),
    assign_colors(SortedCountries, [], Colors).
% Sort countries by number of neighbors (most constrained first)
sort_by_neighbors(Countries, Sorted) :-
    findall(Count-Country, (
        member(Country, Countries),
        count_neighbors(Country, Count)
    ), Pairs),
    sort(0, @>=, Pairs, SortedPairs),
    findall(C, member(_-C, SortedPairs), Sorted).
% Count neighbors for a country
count_neighbors(Country, Count) :-
    findall(N, (adjacent(Country, N); adjacent(N, Country)), Neighbors),
    length(Neighbors, Count).
% Assign colors with early constraint checking
assign_colors([], Acc, Acc).
assign_colors([Country|Rest], Acc, Result) :-
    color(Color),
    \+ conflicts_with_assigned(Country, Color, Acc),
    assign_colors(Rest, [Country-Color|Acc], Result).
% Check if assigning Color to Country conflicts with already assigned countries
conflicts_with_assigned(Country, Color, Assigned) :-
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
    member(OtherCountry-Color, Assigned),
    (adjacent(Country, OtherCountry) ; adjacent(OtherCountry, Country)).
% Helper to find and display a coloring
solve_and_display :-
    find_coloring(Colors),
    write('Valid coloring found:'), nl,
    print_colors(Colors).
% Print the coloring
print_colors([]).
print_colors([Country-Color|Rest]) :-
    write(Country), write(' -> '), write(Color), nl,
    print_colors(Rest).
% Count how many countries use each color
count_color_usage(Colors) :-
    findall(C, member(_-C, Colors), AllColors),
    count_occurrences(AllColors, red, RedCount),
    count_occurrences(AllColors, green, GreenCount),
    count_occurrences(AllColors, blue, BlueCount),
    count_occurrences(AllColors, yellow, YellowCount),
    write('Color usage:'), nl,
    write('Red: '), write(RedCount), nl,
    write('Green: '), write(GreenCount), nl,
    write('Blue: '), write(BlueCount), nl,
    write('Yellow: '), write(YellowCount), nl.
count_occurrences([], _, 0).
count_occurrences([H|T], H, N) :-
    count_occurrences(T, H, N1),
    N is N1 + 1.
count_occurrences([H|T], X, N) :-
    H \= X,
```

SWISH File Edit Examples Help

WXS190012-SXG210250-HW3-Code-Q5

```
    (adjacent(Country, OtherCountry) ; adjacent(OtherCountry, Country)).
% Helper to find and display a coloring
solve_and_display :-
    find_coloring(Colors),
    write('Valid coloring found:'), nl,
    print_colors(Colors).
% Print the coloring
print_colors([]).
print_colors([Country-Color|Rest]) :-
    write(Country), write(' -> '), write(Color), nl,
    print_colors(Rest).
% Count how many countries use each color
count_color_usage(Colors) :-
    findall(C, member(_-C, Colors), AllColors),
    count_occurrences(AllColors, red, RedCount),
    count_occurrences(AllColors, green, GreenCount),
    count_occurrences(AllColors, blue, BlueCount),
    count_occurrences(AllColors, yellow, YellowCount),
    write('Color usage:'), nl,
    write('Red: '), write(RedCount), nl,
    write('Green: '), write(GreenCount), nl,
    write('Blue: '), write(BlueCount), nl,
    write('Yellow: '), write(YellowCount), nl.
count_occurrences([], _, 0).
count_occurrences([H|T], H, N) :-
    count_occurrences(T, H, N1),
    N is N1 + 1.
count_occurrences([H|T], X, N) :-
    H \= X,
    count_occurrences(T, X, N).
```

6.

SWISH File Edit Examples Help

Program Program +

```
% --- Section 1: Facts ---
country_data('Asia', 559056, 19624, 295674, 243758, 5363).
  country_data('Turkey', 126045, 3397, 63151, 59497, 1424).
country_data('Iran', 97424, 6203, 78422, 12799, 2690).
    country_data('China', 82880, 4633, 77766, 481, 33).
country_data('India', 42533, 1391, 11775, 29367, 0).
 country_data('Saudi Arabia', 27011, 184, 4134, 22693, 139).
country_data('Pakistan', 20186, 462, 5590, 14134, 111).
  country_data('Singapore', 18205, 18, 1408, 16779, 22).
 country_data('Israel', 16208, 232, 9749, 6227, 103).

% --- Section 2: Helper Rules ---

sum_list([],0).
sum_list([H|T], Total) :-
    sum_list(T, Rest),
    Total is H + Rest.

% A much clunkier way to find the max value in a list
max_list([H], H).
max_list([H|T], H) :- max_list(T, T_Max), H >= T_Max, !.
max_list([_|T], T_Max) :- max_list(T, T_Max).

% Same clunky logic for the min value
min_list([H], H).
min_list([H|T], H) :- min_list(T, T_Min), H =< T_Min, !.
min_list([_|T], T_Min) :- min_list(T, T_Min).

% A hand-made length function instead of the built-in one
my_length([],0).
my_length([_|T], Len) :-
    my_length(T, LenOfTail),
    Len is LenOfTail + 1.

% --- Section 3: Rules for Answering Corrected Questions ---

% A. Which country has the smallest number of Total cases?
question_a(TheCountry) :-
    findall(Cases, (country_data(C, Cases, _, _, _, _), C \= 'Asia'), AllTheCases),
    min_list(AllTheCases, MinC),
    country_data(TheCountry, MinC, _, _, _, _).

% B. Which country has the largest number of total deaths?
question_b(DaCountry) :-
    findall(D, country_data(_, _, D, _, _, _), DeathList),
    max_list(DeathList, MaxD),
    country_data(DaCountry, _, MaxD, _, _, _).

% C. Which country/countries has/have the largest case number or the smallest number of total deaths?
question_c(Result) :- % part 1
    findall(C, country_data(_,C,_,_,_,_), AllCases),
    max_list(AllCases, MaxVal),
    country_data(Result, MaxVal, _, _, _, _).
```

SWISH File Edit Examples Help

Program Program +

```
    findall(C, country_data(_,C,_,_,_,_), AllCases),
    max_list(AllCases, MaxVal),
    country_data(Result, MaxVal, _, _, _, _).
question_c(Result) :- % part 2
    findall(D, country_data(_,_,D,_,_,_), AllDeaths),
    min_list(AllDeaths, MinVal),
    country_data(Result, _, MinVal, _, _, _).

% D. How many active cases are reported in total for Asia, Turkey, and Iran?
question_d(Total) :-
    country_data('Asia', _, _, _, Active1, _),
    country_data('Turkey', _, _, _, Active2, _),
    country_data('Iran', _, _, _, Active3, _),
    Total is Active1 + Active2+Active3.

% E. Which country/countries has/have more than 1000 serious cases?
question_e(C) :-
    country_data(C, _, _, _, _, Srs),
    Srs > 1000.

% F. Which country has/have more than number of deaths in India?
question_f(CountryName) :-
    country_data('India', _, DeathsInIndia, _, _, _),
    country_data(CountryName, _, OtherDeaths, _, _, _),
    CountryName \= 'India',
    OtherDeaths > DeathsInIndia.

% G. Which country has/have less than number of deaths in India?
question_g(SomeCountry) :-
    country_data('India', _, IndDeaths, _, _, _),
    country_data(SomeCountry, _, CtryDeaths, _, _, _),
    SomeCountry \= 'India',
    CtryDeaths < IndDeaths.

% H. What is sum of the serious cases in this report?
question_h(SumValue) :-
    findall(S, country_data(_,_,_,_,_,S), ListOfSerious),
    sum_list(ListOfSerious, SumValue).

% I. What is the average of death in this report?
question_i(Avg) :-
    findall(Deaths, country_data(_, _, Deaths, _, _, _), AllDeaths),
    sum_list(AllDeaths, Sum),
    my_length(AllDeaths, Count),
    Avg is Sum / Count.

% J. Which country/countries has/have done less than 100k total cases?
question_j(C) :-
    country_data(C, TC, _, _, _, _),
    C \= 'Asia',
    TC < 100000.

% K. Which country/countries has/have done less than 100k and grater than 50 total cases?
```

SWISH File Edit Examples Help

Program Program +

```
    C \= 'Asia',
    TC < 100000.

% K. Which country/countries has/have done less than 100k and grater than 50 total cases?
question_k(Country) :-
    country_data(Country, Cases, _, _, _, _),
    Country \= 'Asia',
    Cases < 100000,
    Cases > 50.
Quoted atom

% L. Which country/countries has/have reported no deaths?
question_l(Country) :-
    country_data(Country, _, 0, _, _, _).

% M. Sort the counties based on the number of serious cases in ascending order.
question_m(Sorted) :-
    findall(Num-C, country_data(C,_,_,_,_,Num), PairList),
    keysort(PairList, Sorted).


% --- Section 4: Main Rule to Show All Answers ---

show_all_answers :-
    writeln('Q A: Which country has the smallest number of Total cases?'),
    question_a(AnsA), write('A A: '), writeln(AnsA),
    writeln('Q B: Which country has the largest number of total deaths?'),
    question_b(AnsB), write('A B: '), writeln(AnsB),
    writeln('Q C: Which country/countries has/have the largest case number or the smallest number of total deaths?'),
    findall(Res, question_c(Res), ListC), write('A C: '), writeln(ListC),
    writeln('Q D: How many active cases are reported in total for Asia, Turkey, and Iran?'),
    question_d(AnsD), write('A D: '), writeln(AnsD),
    writeln('Q E: Which country/countries has/have more than 1000 serious cases?'),
    findall(Res, question_e(Res), ListE), write('A E: '), writeln(ListE),
    writeln('Q F: Which country has/have more than number of deaths in India?'),
    findall(Res, question_f(Res), ListF), write('A F: '), writeln(ListF),
    writeln('Q G: Which country has/have less than number of deaths in India?'),
    findall(Res, question_g(Res), ListG), write('A G: '), writeln(ListG),
    writeln('Q H: What is sum of the serious cases in this report?'),
    question_h(AnsH), write('A H: '), writeln(AnsH),
    writeln('Q I: What is the average of death in this report?'),
    question_i(AnsI), write('A I: '), writeln(AnsI),
    writeln('Q J: Which country/countries has/have done less than 100k total cases?'),
    findall(Res, question_j(Res), ListJ), write('A J: '), writeln(ListJ),
    writeln('Q K: Which country/countries has/have done less than 100k and grater than 50 total cases?'),
    findall(Res, question_k(Res), ListK), write('A K: '), writeln(ListK),
    writeln('Q L: Which country/countries has/have reported no deaths?'),
    ( findall(C, question_l(C), L), L \= [] ->
        write('A L: '), writeln(L)
    ;
        writeln('A L: No country in the report has zero deaths.')
    ),
    writeln('Q M: Sort the counties based on the number of serious cases in ascending order.'),
    question_m(AnsM), write('A M: '), writeln(AnsM).
```

Q A: Which country has the smallest number of Total cases?
A A: Israel

Q B: Which country has the largest number of total deaths?
A B: Asia

Q C: Which country/countries has/have the largest case number or the smallest number of total deaths?
A C: [Asia, Singapore]

Q D: How many active cases are reported in total for Asia, Turkey, and Iran?
A D: 316054

Q E: Which country/countries has/have more than 1000 serious cases?
A E: [Asia, Turkey, Iran]

Q F: Which country has/have more than number of deaths in India?
A F: [Asia, Turkey, Iran, China]

Q G: Which country has/have less than number of deaths in India?
A G: [Saudi Arabia, Pakistan, Singapore, Israel]

Q H: What is sum of the serious cases in this report?
A H: 9885

Q I: What is the average of death in this report?
A I: 4016

Q J: Which country/countries has/have done less than 100k total cases?
A J: [Iran, China, India, Saudi Arabia, Pakistan, Singapore, Israel]

Q K: Which country/countries has/have done less than 100k and grater than 50 total cases?
A K: [Iran, China, India, Saudi Arabia, Pakistan, Singapore, Israel]

Q L: Which country/countries has/have reported no deaths?
A L: No country in the report has zero deaths.

Q M: Sort the counties based on the number of serious cases in ascending order.
A M: [0-India, 22-Singapore, 33-China, 103-Israel, 111-Pakistan, 139-Saudi Arabia, 1424-Turkey, 2690-Iran, 5363-Asia]